CSE6140/CX4140 Spring 2026 Project
Minimum Vertex Cover Problem

April 2, 2026

1 Overview

The Minimum Vertex Cover problem is a well-known NP-complete problem with important applications in graph algorithms, network analysis, scheduling, and combinatorial optimization. In this project, you will study the problem through exact, approximation, and local search algorithms, and evaluate their theoretical and experimental performance on real-world and synthetic datasets.

2 Objectives

- Get hands-on experience in solving an intractable problem that is of practical importance
- Implement an exact branch-and-bound algorithm
- Implement approximate algorithms that run in reasonable time and provide high-quality solutions with guarantees
- Implement heuristic algorithms (without approximation guarantees)
- Develop your ability to conduct empirical analysis of algorithm performance on datasets, and to understand the trade-offs between accuracy, speed, etc., across different algorithms
- Develop teamwork skills while working with other students

3 Groups

You will work in a group of up to 4 students. We encourage you to form your own groups and use Ed Discussion / Piazza as a resource. Afterwards, please designate one member per group to complete the assignment "Group and programming language submission for project" through Canvas by **Mar. 30, Monday, 11:59pm**.

In the text entry, put each student's last name and first name per line, followed by the programming language (Python, C/C++ or Java) your group will use. For instance:

```
Lobo, Macrina
Liu, Bei
Fan, Aaron
Zhu, Shizhuo
Python
```

Note that if your team has fewer than 4 students, we may add students to your team. Also, rubrics will be the same for all teams regardless of the number of members in each team. A team can be formed with students from different sections, eg. graduate section and undergrad section, on-campus sections and DL section.

4 Background

Given an undirected graph

$$G = (V, E),$$

a *vertex cover* is a subset of vertices

$$C \subseteq V$$

such that for every edge

$$(u, v) \in E,$$

at least one of u or v belongs to C. In other words, every edge must be "covered" by one of its endpoints.

The **Minimum Vertex Cover** problem asks us to find a vertex cover of minimum cardinality. Formally, the goal is to find

$$C^* = \arg\min_{C \subseteq V} |C|$$

subject to

$$\forall (u, v) \in E, \;\; u \in C \text{ or } v \in C.$$

For more details on the Vertex Cover problem, refer to Karp's original 21 NP-Complete problems: `https://cgi.di.uoa.gr/~sgk/teaching/grad/handouts/karp.pdf`.

5 Algorithms

Your team will implement **four** algorithms that fall into three categories:

1. **Exact Branch-and-Bound algorithm.**

   Hint: The naive approach would enumerate all subsets of vertices and test whether each is a valid cover. Can we explore the search space more intelligently by branching on decisions and pruning subproblems that cannot improve the best solution found so far? How do we derive useful lower and upper bounds?

2. **Approximation algorithm with approximation guarantees.**

   Hint: There is a simple greedy algorithm that repeatedly selects an uncovered edge and adds both of its endpoints to the cover. This produces a feasible solution with a provable approximation ratio.

3. **Local search.**

   Hint: While local search does not guarantee the convergence of solutions, most of the time it should converge relatively quickly given a good search space. How do you formulate the search space such that the exploration is efficient and doesn't get stuck? This can have a night-and-day difference in simulated annealing or hill climbing algorithms.

In what follows, we present the high-level idea behind the algorithms you will implement

1. **Exact algorithm using Branch-and-Bound**. Implement the Branch-and-Bound algorithm as seen in class.

2. **Approximation algorithm with approximation guarantees**. Please implement an approximation algorithm for Minimum Vertex Cover with approximation guarantees as shown in class.

3. **Local Search**. There are many variants of local search (LS) and you are free to select which one you want to implement. Please implement 2 types/variants of local search. They can be in different families of LS such as Simulated Annealing vs Genetic Algorithms vs Hill Climbing, or they can be in the same general family but should differ by the neighborhood they are using, or by the perturbation strategy, etc.

6 Data

You will run the algorithms you implement on a collection of graph datasets. The dataset package (`data.zip`) has been uploaded to Canvas under assignment "Project". Please make sure you are using the released version of the dataset package and the current version of this handout.

This dataset comprises three sections. The first section is the *test* dataset, which includes small graph instances together with their optimal solution values and one optimal selection of vertices. These are meant to verify correctness of your implementation. The *small* scale dataset also uses exact optimal values, but may omit the exact optimal vertex sets. The *large* scale dataset is intentionally mixed: it contains real-world graphs, constructed instances with known optimal solutions, and harder random / benchmark graphs for which we provide strong benchmark reference values (best-known solutions). These reference values can be used to assess the quality of your algorithm outputs even when proving optimality is impractical.

Table 1: Dataset Description

Set	**Number of Problems**	**# Vertices**	**# Edges**	**Names**
Test	5	6–15	6–22	test#.in, test#.out
Small Scale	18	25–198	78–2742	small#.in, small#.out
Large Scale	12	800–22963	5451–595000	large#.in, large#.out

Some of the large-scale graph instances used in this project are obtained from the Network Repository [1], which provides a large collection of real-world networks and benchmark graphs commonly used in graph mining, network science, and combinatorial optimization. In particular, `large1` and `large12` are chosen as the two focal local-search analysis instances: `large1` is a large sparse real-world Internet topology graph, while `large12` is a dense DIMACS benchmark graph.

Table 2: Large-Scale Instance Families

Instances	**Family**	**Reference Type**	**Role**
large1, large3, large4	Real-world sparse graphs	Best-known reference	realism / sparse structure
large2	Real-world medium graph	Exact optimum	sanity-check real instance
large5, large6	Constructed dense bipartite graphs	Exact optimum	hard instances with known answers
large7, large8	Synthetic random graphs	Best-known reference	random stress tests
large9, large10, large11, large12	DIMACS benchmark graphs	Best-known reference	classical hard benchmark instances

For each instance, the data is structured as follows.

Input Format: The first line of the input file contains two integers separated by spaces, denoting the number of vertices n and the number of edges m, respectively. Vertices are indexed from 1 to n. Each of the following m lines contains two integers u and v, indicating an undirected edge (u, v).

Reference Output Format: The first line of the reference output is a single integer ans. For `test` and `small` instances, this is the optimal size of a minimum vertex cover. For `large` instances, it is either an optimal value (when known) or a benchmark reference value (best-known solution) for that instance. The second line will appear only for `test#.out`, listing the indices of one optimal set of selected vertices.

An example is shown below.

Example Input	Example Output
5 6 1 2 1 3 2 3 2 4 3 5 4 5	3 2 3 4

7 Code

All code should be clearly structured and well commented. Each source file may include a short description of its purpose. A `README` file is needed to explain how to compile and run your program.

Your program must be runnable from the command line using the following format:

```
mvc -inst <instance> -alg [BnB|Approx|LS1|LS2] -time <cutoff> -seed <random seed>
```

where

- `instance` is the name of the dataset instance (for example `small1`)
- `alg` specifies the algorithm
- `cutoff` is the time limit in seconds
- `seed` is the random seed

The instance name should be provided **without the file extension**. Your program should automatically read the corresponding input file `<instance>.in`.

For example, running

```
mvc -inst small1 -alg BnB -time 600 -seed 1
```

should read the input graph from

```
small1.in
```

7.1 Compilation and Execution on Gradescope

Submissions will be compiled and executed by the Gradescope autograder. Depending on the programming language used, the autograder will compile and run the submitted code using the following commands.

Python Python programs will be executed using

```
python3 mvc.py -inst <instance> -alg <algorithm> -time <cutoff> -seed <seed>
```

For example:

```
python3 mvc.py -inst data/test/test1 -alg BnB -time 10 -seed 1
```

C C programs will be compiled and executed using

```
gcc -O2 *.c -o mvc
./mvc -inst <instance> -alg <algorithm> -time <cutoff> -seed <seed>
```

For example:

```
gcc -O2 *.c -o mvc
./mvc -inst data/test/test1 -alg BnB -time 10 -seed 1
```

C++ C++ programs will be compiled and executed using

```
g++ -O2 *.cpp -o mvc
./mvc -inst <instance> -alg <algorithm> -time <cutoff> -seed <seed>
```

For example:

```
g++ -O2 *.cpp -o mvc
./mvc -inst data/test/test1 -alg BnB -time 10 -seed 1
```

Java Java programs will be compiled and executed using

```
javac *.java
java mvc -inst <instance> -alg <algorithm> -time <cutoff> -seed <seed>
```

For example:

```
javac *.java
java mvc -inst data/test/test1 -alg BnB -time 10 -seed 1
```

Your program must support the command-line interface described in this section. Programs that cannot be compiled or executed using the commands above may fail the autograder.

7.2 Autograder clarification

The autograder checks a limited set of correctness and format requirements. Its purpose is to verify:

- correctness of the returned vertex cover value when exact checking is required
- whether the reported vertex set indeed covers all edges
- compliance with the required command-line interface
- compliance with the output file format

The autograded portion is worth **8 points** in total (2 points each for BnB, Approx, LS1, and LS2). The checks are as follows:

- `BnB`: exact correctness on all instances in the *test* dataset
- `Approx`: validity of the returned vertex cover on **all** released *test*, *small*, and *large* instances
- `LS1`: reaching the value given in the corresponding `.out` file on **all** *test* and *small* instances, with a cutoff of **2 seconds** per run
- `LS2`: reaching the value given in the corresponding `.out` file on **all** *test* and *small* instances, with a cutoff of **2 seconds** per run

For the autograded runs, your implementation should produce the required outputs within the stated cutoff times.

The autograder may invoke your program with arbitrary random seeds. Therefore your implementation must remain correct regardless of the seed value.

For local search algorithms, different seeds may lead to different solutions, but the returned vertex set must always be a valid vertex cover.

Again: Submissions that do not follow the required command-line interface or output file naming conventions may fail the autograder.

7.3 Output files

Each run of your program must produce the following output files.

Solution file

Filename format

```
<instance>_<algorithm>_<cutoff>.sol
<instance>_<algorithm>_<cutoff>_<seed>.sol
```

where

- `instance` is the dataset name (for example `small1`)
- `algorithm` is `BnB`, `Approx`, `LS1`, or `LS2`
- `seed` is included only for randomized algorithms

Examples

```
small1_BnB_600.sol
small1_Approx_600.sol
small1_LS1_600_4.sol
small1_LS2_600_7.sol
```

File contents
line 1: size of the vertex cover
line 2: vertices included in the cover
Example:

```
12
1 4 7 9 10 11 15 18 19 20 22 23
```

The autograder will check whether the reported vertex set indeed covers all edges in the graph.

Trace file

Trace files record improvements in the best solution over time.

Filename format

```
<instance>_<algorithm>_<cutoff>.trace
<instance>_<algorithm>_<cutoff>_<seed>.trace
```

Examples

```
small1_BnB_600.trace
small1_LS1_600_4.trace
```

File format
Each line contains, separated by spaces,

1. timestamp in seconds
2. best solution size found so far

Example:

```
0.12 41
0.87 38
2.31 35
```

The number of decimal places in the timestamps of the trace file is not important. Use any precision that is convenient for producing your plots.

Trace files are required for `BnB`, `LS1`, and `LS2`, but are not required for the approximation algorithm.

Important notes on randomness and reproducibility.

- Your implementation should be reproducible when the same seed is used.
- Both local search algorithms should involve randomness and should make use of the provided random seed.
- You may use the approximation algorithm to generate an initial solution for local search and / or to initialize the upper bound of Branch-and-Bound.

8 Report

8.1 Formatting

You will use the format of the Association for Computing Machinery (ACM) Proceedings to write your report. See `http://www.acm.org/publications/proceedings-template` for LaTeX and Word downloads. You may choose any template from this website.

8.2 Content

Your report will account for the majority part of your project points ($36 \sim 40$ points out of 50). It should be written as if it were a research paper submitted to a conference or journal. If you use an algorithm or idea from a paper, textbook, or online resource, please cite it properly. A sample report outline is as follows:

- Abstract
- Introduction: a short summary of the problem, your approaches, and the results you obtained
- Problem definition: a formal definition of Minimum Vertex Cover
- Related work: a survey of existing work on the same problem and important theoretical / practical results
- Algorithms: a detailed description of each algorithm you implemented, including pseudo-code, approximation guarantee (if any), time and space complexity, and design choices. For local search, explain why you chose your search framework, neighborhood, acceptance rule, and parameter settings. Describe any novelty of your methods (if any), and what are the advantages of your methods.
- Empirical evaluation: a detailed description of your platform (CPU, RAM, language, compiler / interpreter, etc.), experimental procedure, evaluation criteria, and results (plots, tables, etc.)
- Discussion: a comparative analysis of how the different algorithms perform with respect to runtime, solution quality, scalability, and the expected theoretical behavior
- Conclusion
- A small number of bonus points may be given for: conducting a thorough literature review to identify advanced state-of-the-art (SOTA) methods in algorithm design or incorporating significant methodological novelty; generating additional informative plots that help illustrate and interpret the results; or performing substantially more experimental runs than required in order to produce smoother and more reliable QRTD and SQD curves.

8.3 Evaluation

We now describe how you will use the outputs produced by your code in order to evaluate the performance of the algorithms.

1. **Comprehensive Table**

   Include a table with columns for each of your Vertex Cover algorithms as shown below. For all algorithms, report the relative error with respect to the reference solution quality provided in the instance files.

   Since this is a minimization problem, relative error ($RelErr$) is computed as

   $$RelErr = \frac{Alg - Ref}{Ref},$$

   where Alg is the size of the solution returned by your algorithm and Ref is the value given in the corresponding `.out` file. For `test` and `small`, Ref is the optimal value; for `large`, Ref may be either an optimal value or a best-known benchmark value, depending on the instance. In other words, when reporting relative error on a `large` instance, use the number written in that instance's provided `.out` file as the baseline.

   Round runtime and $RelErr$ to two digits beyond the decimal point.

   For local search algorithms, your results for each cell should be the average of at least 20 runs with different random seeds for that dataset. You should report average time and average solution size. You may also report any other information you find interesting.

   The comprehensive table should include all instances from `small#` and `large#`. Including `test#` is optional.

	Branch and Bound			Approx / LS1 / LS2		
Dataset	Time (s)	cover size	RelErr	Time (s)	cover size	RelErr
small1	1.14	12	0.00	0.25	13	0.08

Branch-and-Bound can take a long time on large instances. You may use the solution quality obtained by your approximation algorithm as an initial upper bound, which is often much tighter than a naive bound. You may also set a time limit of a few minutes so that your BnB algorithm can still return meaningful results on larger instances.

The next three evaluation plots are applicable to local search algorithms only, and need only be presented for instances `large1` and `large12`. These two instances were selected to represent two contrasting regimes for local search: a large sparse real-world network and a dense benchmark graph. All the information needed to produce these plots is contained in your solution trace files. **Twenty runs** for each local search algorithm are required for the following plots.

1. **Qualified Runtime Distributions (QRTDs)**

   A plot similar to those introduced in lecture on empirical analysis. The x-axis is runtime in seconds, and the y-axis is the fraction of runs that have "solved" the problem.

   Here, solving is defined with respect to some relative solution quality q^*. For example, for $q^* = 1.0\%$, a point with x-value 5 and y-value 0.6 means that in 60% of runs, the algorithm was able to obtain a vertex cover of size at most

   $$Ref \cdot (1 + 0.01)$$

   within 5 seconds, where Ref is the value reported in the provided `.out` file for that instance.

   Varying q^* for several values gives curves similar to those shown in class.

2. **Solution Quality Distributions (SQDs)**

   Instead of fixing the target relative solution quality and varying runtime, now fix runtime and vary solution quality. The details are analogous to those of QRTDs.

3. **Box plots for running times**

   Since local search algorithms are randomized, there can be substantial variation in the time required to reach a target quality. Use box plots, as described in the "Theory" section of this blog post: `http://informationandvisualization.de/blog/box-plot`

   Read the blog post carefully and understand the purpose of this plot type. You may use an online box plot generator if you wish.

9 Deliverables

Failure to abide by the file naming conventions and folder structure detailed here may result in penalties.

Only one student from each group should submit the following:

1. **Report (Gradescope).**

   A PDF file of the report following the guidelines in Section 8. The report should be submitted to Gradescope under the assignment "CourseProject-report".

2. **Programming Autograder (Gradescope).**

   The programming component will be partially evaluated by a Gradescope autograder. The programming component should be submitted to Gradescope under the assignment "CourseProject-program". The autograder is worth **8 points in total** (2 points each for `BnB`, `Approx`, `LS1`, and `LS2`). The autograder checks: `BnB` on the *test* instances, `Approx` for validity on **all** released *test*, *small*, and *large* instances, and `LS1`/`LS2` on whether they reach the value in the corresponding `.out` file on **all** *test* and *small* instances with a 2-second cutoff per run, while also enforcing the required command-line interface and output format.

3. **Code and Output Files (Canvas).**

   Submit a zip file containing:

   - all source code files
   - the executable or scripts used to run the program
   - a `README` describing the structure of the code and how to run the program
   - all output files (`.sol` and `.trace`) generated for the required datasets

   P.S.: DO NOT SUBMIT THE INPUT DATA FILES.

   All files may be placed in the same directory inside the zip archive.

   The zip file should be submitted on Canvas under the assignment "CourseProject-code".

10 Team Evaluation

Each student should also submit a *private* evaluation of their team members. This assignment will be announced later, close to the end of the project. For each team member (including yourself), include a score from 1 to 9 (scores across all team members should sum to 10), a short description of the member's contributions, and a justification for the score. You may skip this assignment if you believe all team members contributed equally.

References

[1] Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization. In *AAAI*, 2015.